Submission Worksheet

IT490-450-M2026 / it490-centralized-logging-milestone

Submission Data

Course	IT490-450-M2026
Assignment	IT490 Centralized Logging Milestone
Student	Branden B. (bb449)
Status	submitted
Worksheet Progress	100%
Potential Grade	10.00/10.00 (100.00%)
Received Grade	0.00/10.00 (0.00%)
Started	6/29/2026, 11:29:35 PM
Updated	6/29/2026, 11:44:13 PM

Grading Link
https://courses.ethereallab.app/assignment/v3/IT490-450-M2026/it490-centralized-logging-milestone/grading/bb449

View Link
https://courses.ethereallab.app/assignment/v3/IT490-450-M2026/it490-centralized-logging-milestone/bb449

Instructions

1. Read the full assignment before starting. The `Freedom and Strict Requirements` section defines the non-negotiable constraints.
2. This is a group assignment. Only one person needs to submit it to Learn/Canvas, but every member must own clear work and appear in the contribution records.
3. Work from a dedicated branch in the group repository, such as `centralized-logging`.
4. Use GitHub Issues to plan and track the milestone. Split work as needed, assign owners by name, GitHub username, and UCID; and keep issues linked to related Pull Request(s).
5. Build the centralized logging workflow described below: app, RabbitMQ, database, and API VM roles; role-specific setup scripts; MQ-based additive log events; listener log files; MQ-only synchronization; and a small dead-letter/error queue path for bad log messages.
6. Submit focused evidence for each task. GitHub links are for full review, while screenshots document the current important code snippets, runtime behavior, command output, and configuration at submission time.
7. If the same evidence already proves part of a later task, reference it instead of uploading it again. Only add new evidence when it proves a new requirement.
8. When a response includes person-specific reflections, decisions, and explanations, include that team member's name with the response.
9. For URL responses, use direct GitHub links to the relevant issue, Pull Request(s), file, and script requested by that task. Do not submit only the repository home page. If a URL task asks for issue tracking, include at least one issue link.
10. Use `N/A` only in contribution records for a member with no contribution, and still include the issue link(s) assigned to that member.

Freedom and Strict Requirements

Teams have implementation freedom, but the core architecture and team-management process are non-negotiable.

Freedom and Strict Requirements

Teams have implementation freedom, but the core architecture and team-management process are non-negotiable.

Technical freedom:

- Teams may choose their VM provider/tooling, API language/runtime, exchange/queue naming, message format details, and script organization.
- Teams may split the implementation into more issues, scripts, files, services, and helper programs when that makes the work clearer.
- Teams may add extra logging message types and extra listeners if the required behavior still works.

Technical restrictions:

- The required roles must exist as separate development VM roles: app, RabbitMQ, database, and API.
- Required VMs must use Ubuntu Server Minimal 24.04 with 1 GB RAM and 1 CPU.
- Each VM must install only the minimum software needed for its role.
- Log synchronization must use RabbitMQ/MQ messages only.
- Do not synchronize logs with shared folders, direct SSH copies, rsync, HTTP calls, database writes, manual copy/paste, and any other VM-to-VM sync path.
- Send additive log message events through MQ. Do not resend and do not synchronize entire log files.
- The MQ topology must include a dead-letter queue (DLQ) or clearly named error queue for malformed or unprocessable log messages. Bad messages must not be silently dropped or endlessly requeued.
- Log entries must clearly identify the source system/server and the logged content.
- Each required listener/VM must maintain a mostly mirrored log file for the message types it consumes.
- The logging publisher system must expose a clearly named application-callable logging interface that future application code can call to publish log events.
- Setup scripts must be role-specific, committed to the group repository, and maintained.

Team-management restrictions:

- The work must be split into GitHub Issues during planning and implementation, not reconstructed only at the end.
- Each issue must have a clear owner and scope. Issue owners must be identified by name, GitHub username, and UCID.
- Every group member must have an individual contribution record with assigned issue link(s), even if the entry is `N/A` because no contribution was done by the team member.
- The final submission must connect issues, Pull Request(s), scripts/code, and member contribution records clearly enough to audit who owned which work.

Submission Progress 100%

Section #1: Milestone Implementation

100%

100%

ⓘ Details

Document the development VMs and confirm each role uses Ubuntu Server Minimal 24.04 with 1 GB RAM, 1 CPU, and the minimum required software installed.

100%

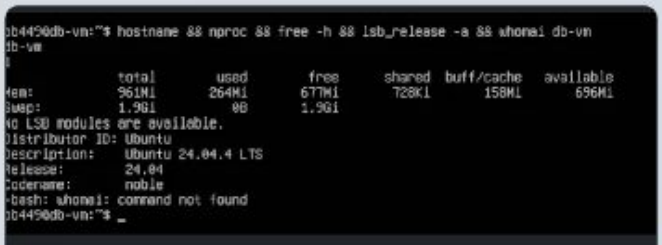
100%

ⓘ Details

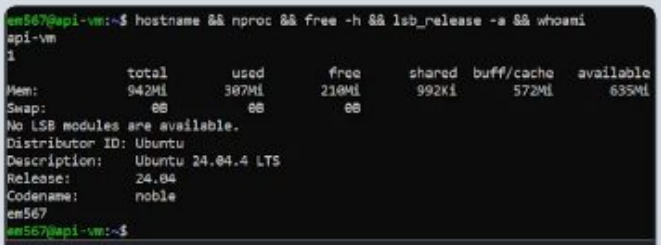
- Provide one concise evidence capture per VM showing Ubuntu Server Minimal 24.04, 1 GB RAM, 1 CPU, hostname, logged-in UCID user, and the role's key installed program/service.
- Captions must identify the VM role, owner, hostname.

Caption 1: APP VM config and apache running

Caption: RabbitMQ VM running with RabbitMQ messaging Server active.



Caption: db-vm baseline installs Owner: Branden (bb449), Hostname:db-vm



Caption: "API VM - owner:Eduardo(em567), hostname: api-vm, Ubuntu 24.04.4 LTS, 1 CPU, 1GB RAM"

100%

ⓘ Details

- Required: link the GitHub issue(s) used to track VM creation and baseline installs.
- The linked issue(s) must show owner and scope for this slice of work.

https://github.com/MattToegel/it490-m2026-racingdevs/issues/9

https://github.com/MattToegel/it490-m2026-racingdevs/issues/11

https://github.com/MattToegel/it490-m2026-racingdevs/issues/7

https://github.com/MattToegel/it490-m2026-racingdevs/issues/5

100%

ⓘ Details

Create separate setup scripts for each VM role. Commit each script to the group repository and keep it maintained.

100%

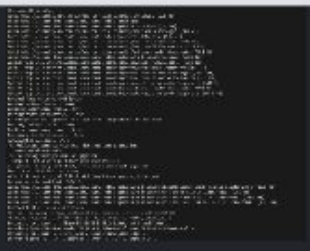
100%

ⓘ Details

- Provide focused screenshots of the relevant setup script snippets for each VM to prove role-specific setup behavior.
- Provide a concise terminal capture showing each setup script completing successfully.
- Captions must identify the VM role, script owner, and file name. Do not repeat the VM inventory screenshots from the previous task.

Caption 1: "API VM setup_api.sh script contents - owner: Eduardo(em567)"

Caption 2: "API VM setup script running successfully - owner: Eduardo(em567)"



Db vm setup_db.script successful install, Owner: Branden (bb449),

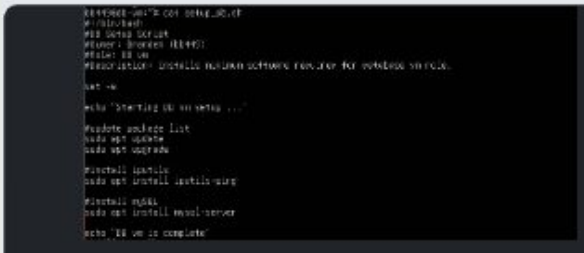
Db vm setup_db.script successful install, Owner: Branden (bb449),),

Caption 1: App vm, Ruchir Patel, app_setup_script.sh

Caption 2: App vm, Ruchir Patel, app_setup_script.sh running successfuly

Caption: RabbitMQ VM setup.sh script contents owner: Michelle (mg792)

Caption: RabbitMQ setup.sh successful execution owner: Michelle (mg792)



DB vm setup_db.sh script contents, Owner Branden (bb449), Hostanem:db-vm

100%

Part 2: URLs 100%

Details

- Required: link directly to each role-specific setup script in the group repository.
- Required: link the issue(s) that track the setup-script work and show owner and scope.
- Required: link the Pull Request(s) that introduced and updated those scripts.

URL #1
https://github.com/MattToegel/it490-m2026-racingdevs/issues/9

URL #2
https://github.com/MattToegel/it490-m2026-racingdevs/issues/11

URL #3
https://github.com/MattToegel/it490-m2026-racingdevs/issues/5

URL #4
https://github.com/MattToegel/it490-m2026-racingdevs/issues/7

URL #5
https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/db/setup_db.sh

URL #6
https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/app/app_setup_script.sh

URL #7
https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/api/setup_api.sh

URL #8
https://github.com/MattToegel/it490-m2026-racingdevs/pull/26

URL #9
https://github.com/MattToegel/it490-m2026-racingdevs/pull/12

URL #10
https://github.com/MattToegel/it490-m2026-racingdevs/pull/24

URL #11
https://github.com/MattToegel/it490-m2026-racingdevs/pull/13

URL #12
https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/setup.sh

Part 3: Response 100%

Details

- Explain what each script installs/configures.
- Explain how the team made the scripts safe to rerun.
- Note which team member owned each script.

Response

Branden: DB setup script The script installed for the DB VM role installs mysql to create, manage and store the databases. Iputils was also installed.

Ruchir Patel: App setup script The script for the APP VM role installs Apache, php, composer, iputils-ping, and nano. Apache is used for its web-server capabalities with php to run server-side code. Composer is used to manage php dependcies and iputils to test network connectivity. nano is used to to as a command line editor.

Eduardo Manticorena: API setup Script The Script installed for the API VM role were Python3 for the runtime used for the API service and python3-pika for the RabbitMQ client library used to consume and publish log messages over MQ. The script is safe to rerun because it checks if Python3 is already installed before it

used for the API service and python3-pika for the RabbitMQ client library used to consume and publish log messages over MQ. The script is safe to rerun because it checks if Python3 is already installed before it attempts to install it and checks if python3-pika is installed already using dpkg before attempting to install again. So if either are installed, the script skips that step and prints a confirmation message and no duplicate installs will occur on repeated runs.

Michelle Gonzalez: Rabbitmq Setup Script The sscript installed for the Rabbitmq role were rabbit-mq server to host and start the rabbit mq-server. It also downloads composer and moves it to the apprporiate area to install it.

Supports **bold**, *italic*, [links](url), `code`, etc. 1290 chars

Task #3 (1 pt.) – MQ Message Contract and Routing Design

100%

Details

Design the MQ message shape, routing behavior, and dead-letter/error queue handling before implementing the publishers and listeners.

Combo #1

Part 1: URLs 100%

Details

- Required: link the issue that tracks the message contract and MQ topology work.
- Required: link the source file and config that define exchanges, queues, routing keys, fanout behavior, relay behavior, and dead-letter/error queue behavior.
- The linked issue(s) must show owner and scope for this slice of work.

URL #1
https://github.com/MattToegel/it490-m2026-racingdevs/issues/10

URL #2
https://github.com/MattToegel/it490-m2026-racingdevs/issues/7

URL #3
https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/mq/log_message_contract.json

URL #4
https://github.com/MattToegel/it490-m2026-racingdevs/pull/14

URL #5
https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/api/config.py

URL #6
https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/mq/log_message_contract.json

Part 2: Response 100%

Details

- Explain the message fields.
- Explain how message type maps to consumers/listeners.
- Explain how the design supports additive log messages without syncing entire log files.
- Explain what happens when a log message is malformed or cannot be processed.

Response

Response

Message Fields: The message field identifies the vm that generates the log message. It adds a timetsap and puts a a level of importance and condition of the message log. Lastly it generates a string of the message that was sent.

Message type: Log level is used to determine importance through info, warning, or error. The exchange matches the route key making sure it matches the consumer.

Design: Log messages are handled individually by the rabbitmq. That individual log message is then handled by the consumer. Instead of of a entire log file it is handled indvidually.

Malformed log Message: The required fields are first validated first and if any are found to be invalid, that message is sentto DLQ.

Supports **bold**, *italic*, [links](url), `code`, etc. 710 chars

Task #4 (1 pt.) – Publishers, Consumers, and Log Relay

100%

ⓘ Details

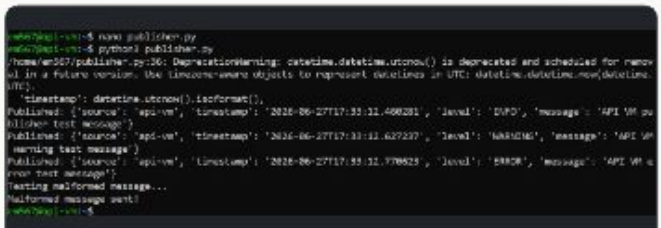
Implement the publishers and consumers/listeners that synchronize log messages through MQ. The publishing side must expose a clearly named application-callable logging interface that future application code can call when it needs to publish a log event. Consumers/listeners must route malformed or unprocessable log messages to the DLQ/error queue instead of silently discarding or endlessly requeueing them.

Combo #1

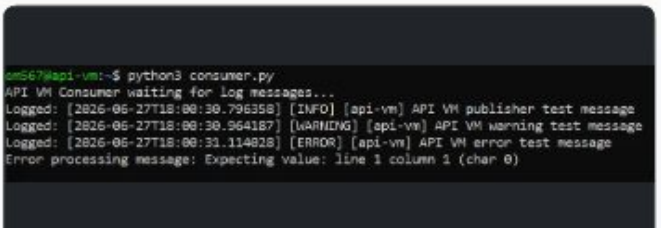
Part 1: Images 100%

ⓘ Details

- Provide focused screenshots of the publisher, the exposed application-callable logging interface intended for future application code, consumer/listener, and log append snippets.
- Captions must identify the source VM and listener role, message type, file name, and log file being updated.
- Runtime proof belongs in the end-to-end demo task unless the same screenshot is needed to explain the code.



source:api-vm, (info,warning,error) publisher.py

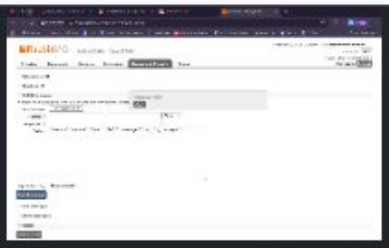


source:api-vm, (info,warning,error) consumer.py

Configured log_queue with dead letter exchange (log_exchange) and routing key (dlq) for failed message handling Owner: Michelle

Caption: RabbitMQ log_queue successfully created for centralized logging

handling Owner: Michelle



Caption: Test log message successfully published to log-queue using the rabbitmq management interface Owner: Michelle (mg792)

Successfully verifies that published test message was stored in log_queue and failed message routed to log_queue_dlq, Michelle

Successfully verifies that published test message was stored in log_queue and failed message routed to log_queue_dlq, Michelle

Caption 2: running a test on the logger code

Part 2: URLs 100%

ⓘ Details

- Required: link the publisher code, exposed application-callable logging interface, consumer/listener code, and log append code in the group repository.
- Required: link the issue(s) that track this implementation and show owner and scope.
- Required: link the Pull Request(s) that introduced and updated this work.

URL #1
https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/api/publisher.py

URL #2
https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/api/consumer.py

URL #3
https://github.com/MattToegel/it490-m2026-racingdevs/issues/19

URL #4
https://github.com/MattToegel/it490-m2026-racingdevs/issues/21

URL #5
https://github.com/MattToegel/it490-m2026-racingdevs/pull/20

URL #6
https://github.com/MattToegel/it490-m2026-racingdevs/pull/22

URL #7
https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/api/config.py

URL #8
https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/app/log_config.php

URL #9
https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/app/publish_log.php

URL #10
https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/app/route_dlq.php

Part 3: Response 100%

ⓘ Details

- Explain how future application code would call the exposed application-callable logging interface to publish a log event.
- Explain how publishers generate messages.
- Explain how consumers/listeners subscribe to messages.
- Explain how malformed or unprocessable messages are routed to the DLQ/error queue.
- Explain where log files are written and how entries include source system/server and logged content.

Response

In the future, our app code will call the publish_log() function (from publisher.py) and pass it the source VM name, log level, and message text.

The publisher connects to RabbitMQ (using the configured host) and sends the log as a JSON message to the log_exchange with the routing key log.

On the API VM, a consumer service listens to the log_queue. When a message comes in, it validates that it contains all the required fields (source, timestamp, level, and message). If everything checks out, it formats the entry nicely and appends it to the local log file. Every entry clearly shows which VM the log came from and what the message says.

If a message is malformed or missing required fields, the consumer rejects it with basic_nack (and requeue=False), which sends it to the dead-letter queue log_queue_dlq. Bad messages are never silently dropped or stuck in endless retry loops.

All logs are written to /home/em567/api_vm.log on the API VM.

Supports **bold**, *italic*, [links](url), `code`, etc. 952 chars

Task #5 (1 pt.) – MQ-Only Synchronization Verification

100%

ⓘ Details

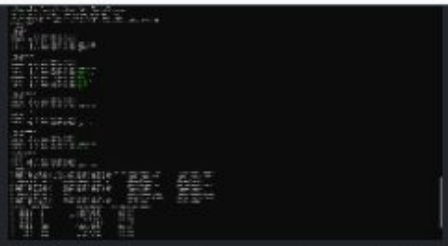
Prove that log synchronization happens only through MQ and not through another VM-to-VM communication path.

Combo #1

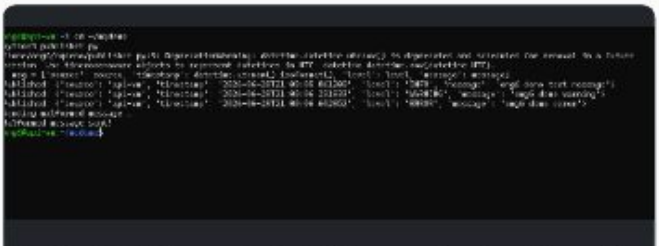
Part 1: Images 100%

ⓘ Details

- Provide focused evidence from the MQ-only verification test. Include command output and any relevant configuration shown during the test.



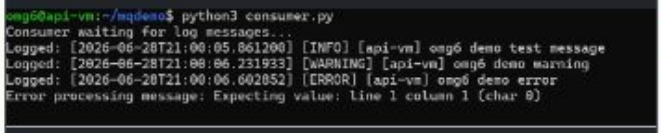
Caption - verify mq-only.sh on api-vm. No NFS/CIFS mounts, no rsync/scp cron jobs, no file-sync timers, and no cross-VM traffic



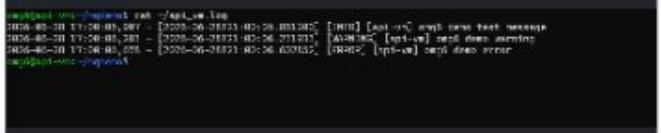
Caption:Publisher sends 3 valid log events + 1 malformed to log_exchange. Owner: Omar (omg6), source: api-vm

Caption - verify mq-only.sh on api-vm. No NFS/CIFS mounts, no rsync/scp cron jobs, no file-sync timers, and no cross-VM traffic

Caption:Publisher sends 3 valid log events + 1 malformed to log_exchange. Owner: Omar (omg6), source: api-vm



Caption: Consumer on api-vm logs the 3 valid events and rejects the malformed one; timestamps match the published events. Owner:



Caption: Mirrored log /home/omg6/api_vm.log — entries show timestamp, level, and source (api-vm). Owner: Omar (omg6)

Part 2: URLs 100%

ⓘ Details

- Required: link the verification issue.
- Required: link the Pull Request(s) related to this verification work.
- The linked evidence must support that none of the following are responsible for log propagation: direct file copy, rsync, shared folder, database write, HTTP call, SSH-based sync.
- The linked issue(s) must show owner and scope for this slice of work.

URL #1

https://github.com/MattToegel/it490-m2026-racingdevs/issues/16

URL #2

https://github.com/MattToegel/it490-m2026-racingdevs/pull/23

URL #3

https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/mq/verify-mq-only.sh

Part 3: Response 100%

ⓘ Details

- Explain the test used to prove MQ-only synchronization.
- Identify any ports/services required for the architecture.

Response

To verify logs sync with RabbitMQ, I had to run two tests. A negative check script that no other path exists, and the only cross-VM traffic is RabbitMQ on port 5672. Then I ran a broker stop test with RabbitMQ. The server stopped, the publisher failed to connect, and nothing happened. This proves RabbitMQ is the only path. The VMs use Tailscale and the network transport.

Supports **bold**, *italic*, [links](url), `code`, etc. 374 chars

Task #6 (2 pts.) – End-to-End Demo and Verification

100%

ⓘ Details

Demonstrate the complete centralized logging workflow from source event to mirrored appended log entries.

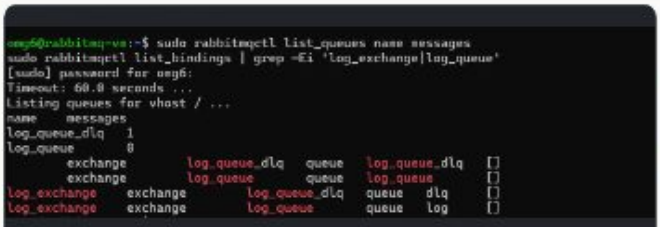
Demonstrate the complete centralized logging workflow from source event to mirrored appended log entries.

Combo #1

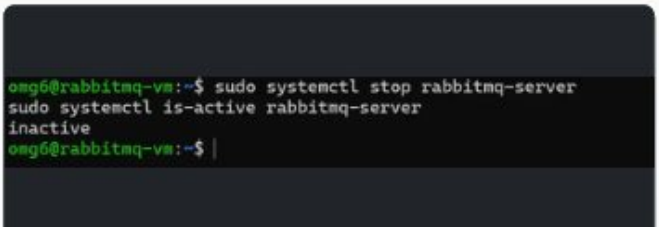
Part 1: Images 100%

Details

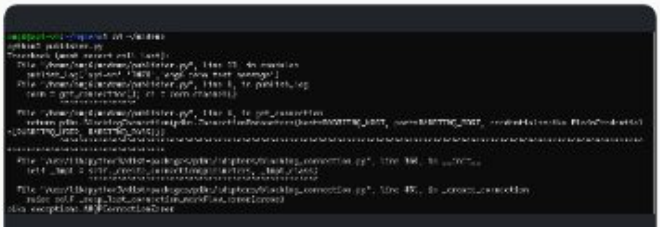
- Provide enough runtime evidence to follow one complete log event from source to appended entries on the expected consumers/listeners.
- Include timestamps and message content that make it clear the entries came from the same additive message event.
- Include one small malformed-message test showing that the message is captured by the DLQ/error queue.
- Captions must explain the source VM, destination listener, and log file. You can reuse earlier evidence when it proves the same requirement.



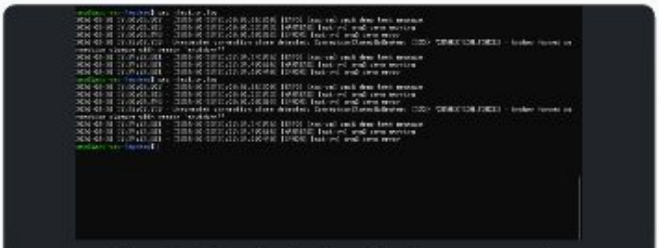
Caption: RabbitMQ shows the malformed message captured in log_queue_dlq (dead-letter queue), Omar (omg6)



Caption:rabbitmq-server stopped on rabbitmq-vm for the MQ-only test. Owner: Omar (omg6)



broker down, the publisher cannot connect (AMQPConnectionError) — no logs propagate, proving MQ is the only sync path. Omar,omg6



Log file showing the broker shutdown event, then propagation resuming after restart. Owner: Omar (omg6)

Part 2: URLs 100%

Details

- Required: link the final demo/verification issue(s).
- Required: link the Pull Request(s) related to the final implementation and verification.
- The linked issue(s) must show owner and scope for the final demo and verification slice.

URL #1

https://github.com/MattToegel/it490-m2026-racingdevs/issues/18

URL #2

https://github.com/MattToegel/it490-m2026-racingdevs/pull/23

Part 3: Response 100%

Details

- Explain the demo path from source log event to MQ message to appended log files.
- Explain the malformed-message test and where the DLQ/error queue captured the bad message.
- Explain what passed, what failed, and what remains incomplete.

Response

The end-to-end demo follows one log event.Publish_log() sends a JSON message to log_exchange, the

Response

The end-to-end demo follows one log event.Publish_log() sends a JSON message to log_exchange, the consumer on api-vm checks the fields, and appends it to /home/omg6/api_vm.log. This matches timestamps between published and logged messages to confirm it's the same event on MQ. The malformed message is rejected and sent to log_queue_dlg instead of being dropped.

Supports **bold**, *italic*, [links](url), `code`, etc. 365 chars

Section #2: (3 pts.) Reflection And Contributions

100%

Task #1 (0.5 pts.) – What was the easiest part of this assignment

100%

ⓘ Details

At least a few thoughtful sentences. This can be a composite of thoughts from each member; label person-specific comments with the team member's name.

Response

The easiest part was setting up the vms and connecting them via tailscale.

Supports **bold**, *italic*, [links](url), `code`, etc. 74 chars

Task #2 (0.5 pts.) – What was the hardest part of this assignment

100%

ⓘ Details

At least a few thoughtful sentences. This can be a composite of thoughts from each member; label person-specific comments with the team member's name.

Response

The hardest part was running the mq test and demo to make sure that everythings was running correctly and as required.

Supports **bold**, *italic*, [links](url), `code`, etc. 118 chars

Task #3 (0.5 pts.) – What did you learn during this assignment

100%

ⓘ Details

At least a few thoughtful sentences. This can include any noteworthy comments from the team; label person-specific comments with the team member's name.

Response

We learned how to setup multiple vms and use them to create a centralized logging system.

Response

We learned how to setup multiple vms and use them to create a centralized logging system.

Supports **bold**, *italic*, [links](url), `code`, etc. 89 chars

Task #4 (1.5 pts.) – Individual Contribution Records

100%

Details

Add one item per group member. Each item must identify the member, summarize their owned work, and link the issues, Pull Request(s), scripts/code, and evidence related to their contribution.

Entry #1

Part 1: URLs 100%

Details

- Required for every member: add this member's assigned GitHub issue link(s).
- For contributing members, add only the Pull Request(s), setup scripts, source files, and evidence links that directly support this member's contribution summary.
- For members with no contribution, include the issue link(s) for the work assigned to them.

URL #1
https://github.com/MattToegel/it490-m2026-racingdevs/pull/12

URL #2
https://github.com/MattToegel/it490-m2026-racingdevs/pull/13

URL #3
https://github.com/MattToegel/it490-m2026-racingdevs/pull/14

URL #4
https://github.com/MattToegel/it490-m2026-racingdevs/pull/8

URL #5
https://github.com/MattToegel/it490-m2026-racingdevs/pull/17

URL #6
https://github.com/MattToegel/it490-m2026-racingdevs/pull/20

URL #7
https://github.com/MattToegel/it490-m2026-racingdevs/pull/22

URL #8
https://github.com/MattToegel/it490-m2026-racingdevs/pull/23

URL #9
https://github.com/MattToegel/it490-m2026-racingdevs/pull/24

URL #10
https://github.com/MattToegel/it490-m2026-racingdevs/pull/25

URL #11
https://github.com/MattToegel/it490-m2026-racingdevs/pull/26

URL #12
https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/api/setup_api.sh

URL #13
https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/app/app_setup_script.sh

URL #14
https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/db/setup_db.sh

URL #15

URL #14
https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/db/setup_db.sh

URL #15
https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/api/config.py

URL #16
https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/api/consumer.py

URL #17
https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/api/publisher.py

URL #18
https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/app/log_config.php

URL #19
https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/app/publish_log.php

URL #20
https://github.com/MattToegel/it490-m2026-racingdevs/blob/main/app/route_dlq.php

Part 2: Response 100%

Details

- Include the group name.
- Include this member's name, GitHub username, and UCID.
- Summarize the member's owned issue(s), completed work, reviews, verification, and any unfinished items.
- Use this task for the member-by-member ownership summary instead of repeating ownership details in each implementation task.
- If a member made no contribution, still add an item for them. The response for that member must say `N/A`, briefly state that the assigned issue work was not completed, and the URL item must still include the issue link(s) assigned to that member.

Response

Group Name: Racing Devs

Ruchir Patel, (ruchirexe), (rkp28):

- Issues: #3, #5, #6,
- Completed Work: App VM steup and Base Install, App VM Logging and publishing interface

Michelle Gonzalez, (michelleegon), (mg792):

- Issues: #7
- Completed Work: Rabbit vm setup and Log Que + DLQ configuration

Eduardo Manticorena, (EduardoM567), (em567):

- Issues: #9, #10, #19, #21
- Completed works: API vm baseline install and setup scripts.MQ Message Contract and Routing Design, Publihsers, Consumers, and Log Relay, Update API vm publisher and consumer to use config file

Omar Gad (omargad-sys), (omg6):

- Issues: #16, #18 -Completed Works: MQ-only Sychronization Verification, End-to-End Centralized Logging Demo and Verification

Branden Brison, (bb449), Stealthraider: Issues: #11 Completed works: Database VM and setup and Baseline Install

Supports **bold**, *italic*, [links](url), `code`, etc. 840 chars

Supports **bold**, *italic*, [links](url), `code`, etc. 840 chars

End of Assignment - submission has been recorded.